MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)
SEM 2 2025/2026

LAB 06:
SMART SURVEILLANCE CAMERA USING ESP32-CAM

SECTION 1
GROUP 10

INSTRUCTOR:
DR ZULKIFLI BIN ZAINAL ABIDIN

Name	Matric Number
NUR IRDINA BINTI OMAR	2312378
MUHAMMAD IRSYAD ILHAM BIN AZIZAN	2217555
NURIN NAJWA BINTI KAMARUZAMAN	2416554

DATE OF SUBMISSION: WEDNESDAY, 15 APRIL 2026

Abstract

The ESP32-CAM module is used in this experiment to create a smart surveillance system. The main objective is to simulate a simple automated monitoring system by combining servo motor control and live video streaming. A servo motor was utilized to allow horizontal camera movement, and the ESP32-CAM was set up to send real-time video over Wi-Fi. To improve system interaction, additional features including face detection and pushbutton control were tested. All things considered, this experiment shows how hardware and software are integrated in a mechatronic system, highlighting the importance of wireless communication, real-time control, and system synchronization.

Table of Contents

Introduction

Smart surveillance systems play an important role for automation, security, and monitoring for modern engineering applications. Simple and affordable devices like the ESP32-CAM have made it simpler to create intelligent monitoring systems with the development of Internet of Things (IoT) technologies. The goal of this experiment is to use the ESP32-CAM module to develop and implement a smart security camera system. The system combines several mechatronic components, such as wireless communication, microcontrollers, actuators, and sensors. A servo motor enables the camera to pan horizontally, which simulates motion tracking, while the ESP32-CAM enables real-time video streaming via Wi-Fi. Additionally, the system can be enhanced with capabilities like face detection for automated tracking and pushbutton control for human operation. So, we can learn how several subsystems cooperate to create a whole, functional mechatronic system through this experiment.

Task 1 : Integrate a pushbutton for manual control of servo panning

2. Material and Equipments

- ESP32-CAM
- Micro-USB Cable
- Servo Motor
- 2 Pushbutton
- Jumper wires
- Breadboard

3. Experimental Setup

In this experiment , the components are connected so that the servo motor and push button can work properly with the ESP32 cam. This setup allows the user to control the movement of the servo manually using a pushbutton.

Servo Motor

- 5V ESP connected to 5V Servo Motor (Red)
- GND ESP connected to GND Servo Motor (Brown)
- IO13 ESP connected to Signal Servo Motor (Yellow)

Push Button

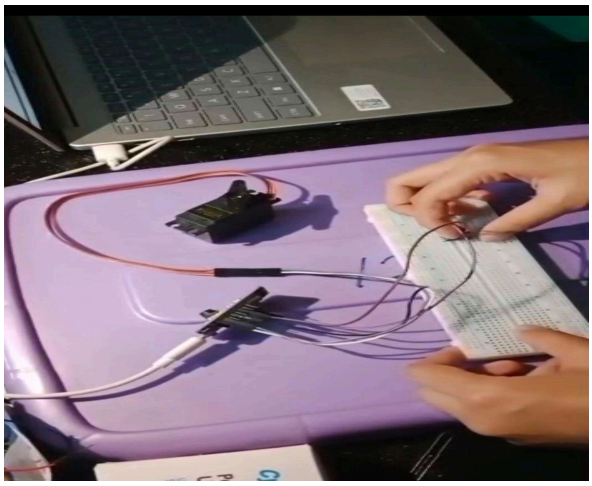
- IO14 ESP → Push Button (one leg)
- GND ESP → Push Button (other leg)
- 

Figure 2.1 : Circuit Setup for Task a Pushbutton for manual control of servo panning

4. Methodology

Hardware Setup

First, all required components such as the ESP32-CAM module, servo motor, pushbutton, breadboard, jumper wires, and micro-USB cable were prepared. The ESP32-CAM was used as the main microcontroller to control the servo motor movement based on the pushbutton input.

Servo Motor Connection

The servo motor was connected to the ESP32-CAM to enable panning movement.

- The red wire of the servo motor was connected to the 5V pin of the ESP32-CAM.
- The brown wire of the servo motor was connected to the GND pin.
- The yellow signal wire of the servo motor was connected to GPIO13.

These connections allowed the ESP32-CAM to send PWM signals to control the rotation angle of the servo motor.

Pushbutton Connection

Next, the pushbutton was connected to provide manual control input.

- One leg of the pushbutton was connected to GPIO14 of the ESP32-CAM.
- The other leg of the pushbutton was connected to GND.

The pushbutton functioned as a trigger switch to control the servo panning movement manually.

Circuit Setup

After all components were connected, the complete circuit was arranged neatly on the breadboard using jumper wires. It was checked carefully to avoid loose connections during the experiment.

Programming the ESP32-CAM

The Arduino IDE software was used to write and upload the program into the ESP32-CAM. The ESP32Servo library was included in the coding to generate PWM signals for servo control.

The program was designed with the following functions:

- Detects the pushbutton input from GPIO14.
- Rotate the servo motor when the pushbutton was pressed.
- Return the servo motor to its original position after the button was released.

Arduino Coding :

```
#include <ESP32Servo.h>


Servo myServo;


int buttonPin = 13;

int servoPin = 14;


int pos = 0;

bool direction = true;


void setup() {

  pinMode(buttonPin, INPUT_PULLUP);

  myServo.attach(servoPin);

}


void loop() {


  int buttonState = digitalRead(buttonPin);
```

```
  if (buttonState == LOW) {


    // Servo panning
    if (direction) {
      pos++;
      if (pos >= 180) direction = false;
    } else {
      pos--;
      if (pos <= 0) direction = true;
    }

    myServo.write(pos);
    delay(10);
  }



}
```

System Testing

Finally, the system was tested several times to observe the servo motor response during pushbutton operation. The pushbutton was pressed repeatedly to verify that the servo motor could perform smooth and accurate panning movement without delay. The experiment successfully demonstrated manual servo panning control using a pushbutton integrated with the ESP32-CAM module.

5. Data Collection

In order to gather data for this experiment, the servo motor's movement was observed when the pushbutton was depressed. The ESP32 used the programmed code to control the servo motor, and the pushbutton served as the input device. To determine whether the servo motor could react appropriately to the pushbutton input, a number of tests were carried out.

The servo motor created a panning movement during the experiment by continuously rotating from 0° to 180° and then reversing back to 0° when the pushbutton was pressed. Throughout the experiment, the servo movement's responsiveness, stability, and smoothness were noted. The servo motor instantly stopped at its current position when the pushbutton was released.

6. Data Analysis

During the experiment, it can be observed that the servo motor was able to react appropriately when the pushbutton was pressed. The input from pushbutton was successfully detected by the ESP32, which then used the programmed code to control the servo motor. The servo produced a continuous panning movement by smoothly moving from 0° to 180° and then back to 0°.

The servo motor moved steadily and responsively throughout the experiment without experiencing any noticeable delays. The servo motor instantly stopped at its current position when the pushbutton was released. This demonstrated the proper and reliable operation of the servo and input control throughout the testing procedure.

7. Result

The experiment successfully demonstrated the ESP32 system allowed the pushbutton to effectively control the servo motor. When the pushbutton was pressed, the servo motor could continuously spin left and right.

Furthermore, the system accomplished the goal of manual servo panning control and ran smoothly. The servo motor was able to react precisely to the pushbutton input because the hardware connections and programming were operating as intended.

Task 2: ESP 32 CAM Face detection and Servo Tracking

2. Material and Equipments

- ESP32-CAM Module version OV2640
- Micro-USB Cable
- Servo Motor
- Pushbutton
- Jumper wires

3. Experimental Setup

1. ESP32-CAM to Servo Motor connections
    - 5V ESP connected to 5V Servo Motor (Red)
    - GND ESP connected to GND Servo Motor (Brown)
    - GPIO13 ESP connected to Signal Servo Motor (Yellow)

2. ESP32-CAM connected to the laptop via USB cable.

3. The GPIO pin bridged to the GND pin (only during uploading the code)

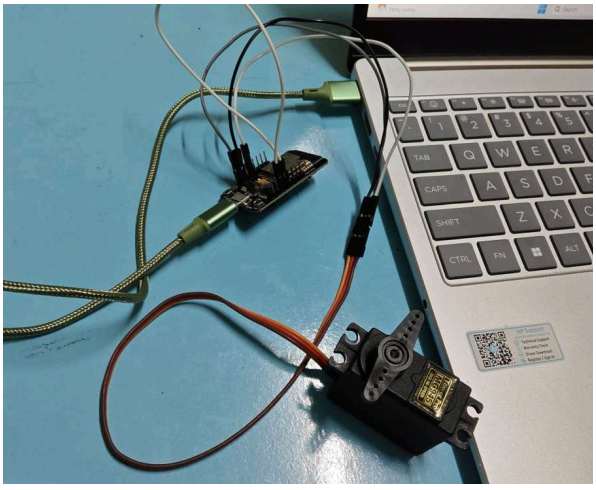


Figure 3.1 Circuit Setup for ESP32-CAM and Servo Motor

4. Methodology

Hardware Setup

The ESP32-CAM was connected to Servo Motor with connection, 5V to 5V (red), GND to GND (brown) and GPIO13 to Signal Servo Pin (yellow). The ESP32-CAM was connected to the laptop via USB to upload the coding and as a power supply. During uploading the code, the GPIO pin needs to connect to the ground temporarily. After the coding was uploaded, GPIO was disconnected from ground.

Programming

CameraWebServer_Servo.ino

The main system initialization which handled the basic setup of the ESP32-CAM system.

```
#include <WiFi.h>
#include <ESP32Servo.h>
myServo.attach(servoPin, 500, 2400);
myServo.write(servoPos);
esp_camera_init(&config);
WiFi.begin(ssid, password);
startCameraServer();
```

This code prepared the camera, servo motor, and Wi-Fi connection so the system could operate as a smart surveillance device.

app_httpd.cpp

The core processing section as it handles the face detection and automatic servo movement.

1. It works as web server initialization which creates the http server for users to access the ESP32-CAM through a browser. Commands that create web server handles for camera control webpage and live video streaming:

   ```
   httpd_handle_t stream_httpd = NULL;
   httpd_handle_t camera_httpd = NULL;
   ```

2. Activate the face detection as the library loads the ESP32's built- in face detection models to detect possible face regions then improve detection accuracy using facial landmarks. This process allows the ESP32–CAM to recognize face boundaries in each frame. It is enabled only when PSRAM is available since image processing requires more memory.

   ```
   #define CONFIG_ESP_FACE_DETECT_ENABLED 1
   #include "human_face_detect_msr01.hpp"
   #include "human_face_detect_mnp01.hpp"
   ```

3. The camera will continuously capture image frames from the video stream. Each frame is then analyzed for face detection before being sent to the browser. The command is:

   ```
   fb = esp_camera_fb_get();
   ```

4. Once a face is detected, the system determines its horizontal center position to determine whether the face is on the left, center, or right side of the screen. This code was inside function draw_face_boxes():

   ```
   x = (int)prediction->box[0];
   w = (int)prediction->box[2] - x + 1;
   int centerX = x + (w / 2);
   ```

   where:
   x = Left boundary of face
   w = Face width
   centerX = Horizontal center of detected face

5. The detected face position is converted into servo movement inside function draw_face_boxes():

   ```
   int angle = map(centerX, 0, fb->width, 180, 0);
   if (angle < 10) angle = 10;
   if (angle > 170) angle = 170;
   myServo.write(angle);
   ```

   The face following movement will be automatic where the servo will rotate to the left if the face was at the left side of screen and vice versa.

camera_index.h

This file works as web interface control. It allowed the user to control streaming and face detection from a web browser without changing the code manually.

<img id="stream"> → Displays live camera feed
startStream() → Starts live streaming
toggleDetect() → Turns on face detection
fetch('/control?var=face_detect&val=1') → Sends command to ESP32-CAM

camera_pins.h

This file acted as hardware configuration which defined hardware pin connection. This ensured the ESP32-CAM hardware was correctly connected and compatible with the software.

#define CAMERA_MODEL_AI_THINKER → Selects AI Thinker ESP32-CAM board
#define Y2_GPIO_NUM ... Y9_GPIO_NUM → Camera data pins
#define XCLK_GPIO_NUM → Camera clock pin
#define LED_GPIO_NUM → Flash LED pin

5. Data Collection

Data was collected by observing system functionality and recording ESP32-CAM performance in wireless streaming, face detection, and servo tracking under real-time conditions. The extracted telemetry string was structured as follows:

$$\text{Data vector} = \{t, x, w, C_x, \Delta E_x, \theta\}$$

Where,

t = System runtime timestamp (ms)
x = Box X
w = Box Width
C_x = Face Center
θ = Servo Angle
ΔE_x = Horizontal tracking error calculated relative to the frame center: 160 pixels

$$\Delta E_x = C_x - 160$$

t (ms)	x	w	C_x	ΔE_x	θ°	observation
14250	38	62	69	-91	141	Face at the left margin; servo rotates left
14390	130	60	160	0	90	Face directly centered in the camera's FOV, servo didn't move
10550	158	64	207	47	64	Face at the right margin; servo rotates right

6. Data Analysis

The collected data was analyzed to evaluate how well the ESP32-CAM and servo motor work together as a closed-loop feedback system. When the face is on the left side, which $C_x < 160$ pixels, the tracking error ΔE_x is negative. The system detects this error and immediately moves the servo to a higher angle, which $\theta > 90°$ to turn the camera to the left. If the face is exactly at the center where $C_x = 160$ pixels, the tracking error becomes zero, and the servo stays at its middle position, 90°. This shows the system has reached equilibrium. Meanwhile, when the face moves to the right side where $C_x > 160$ pixels, the error becomes positive. The servo decreases its angle where $\theta < 90°$ to turn the camera to the right.

7. Results

The experiment successfully achieved its objectives. The ESP32-CAM was able to detect a human face in real-time and automatically move the servo motor to track it. Whenever a face entered the camera's view, the servo motor immediately responded. If the person moved to the left, the servo rotated clockwise to point the camera left. If the person moved to the right, the servo rotated counter-clockwise to follow them. Based on the testing, when the face is not centered, the error is not zero. The servo moves dynamically to catch up with the face. If the face is perfectly in the center, the error becomes zero, and the servo stops exactly at 90°. Lastly, when the person walks away and no face is detected, the servo stops and safely holds its last position without shaking or twitching.

Discussion

This experiment demonstrated the development of a smart wireless surveillance system using an ESP32-CAM and a servo motor. The system highlighted the integration of embedded edge computing, wireless networking, and physical actuation through manual control like in Task 1 and automated face tracking in Task 2.

In Task 1, manual pan control was achieved by programming the ESP32-CAM to scan specific input general-purpose input/output (GPIO) registers like (GPIO14). When a user pressed the pushbutton, the logic level dropped to a LOW state due to the pull-up resistor configuration. The microcontroller processed this input signal to adjust the internal Pulse Width Modulation (PWM) signal transmitted via GPIO13 to the servo motor, altering its physical angle in precise steps. It highlights an open-loop control configuration, where the system relies entirely on human feedback to align the camera with a target object.

In Task 2, the setup was upgraded into an intelligent closed-loop automated tracking framework using a Wi-Fi-hosted camera web server. The ESP32-CAM streamed high-definition video frames to a localized IP address. When the Face Detection function was enabled, the software algorithm calculated the bounding coordinates of the detected human face in real-time. The system computed a continuous horizontal tracking error by designating the horizontal frame center as the target reference point where $\mathbf{C_x} = 160$ pixels. If the face shifted away from the center, a non-zero error triggered an instant mathematical correction in the servo's position registers, forcing the motor to rotate and recenter the target. This loop operated continuously, demonstrating how image extraction calculations can directly drive localized mechanical motion without needing heavy mainframe computers.

However, several technical challenges and potential sources of error were observed during live testing. First, a high network traffic or signal attenuation caused data packet delays, leading to choppy video streams and lag in the servo's movement. Then, the high current draw from the servo during rapid tracking movements occasionally caused the ESP32-CAM to reset due to power instability. Lastly, low ambient light or strong shadows altered the facial contrast, causing the camera to lose track of the face.

Conclusion

In conclusion, this experiment successfully demonstrated the development and implementation of a smart wireless surveillance system by integrating an ESP32-CAM module with a servo motor. Through the manual control setup,which in Task 1, it was verified that physical pushbutton triggers can reliably alter digital GPIO registers to actuate the servo motor in precise mechanical steps. Meanwhile, in Task 2, Through the manual control setup, it was verified that physical pushbutton triggers can reliably alter digital GPIO registers to actuate the servo motor in precise mechanical steps. The ESP32-CAM dynamically updated its PWM output registers by calculating real-time mathematical horizontal tracking errors of a detected face to control the servo and recenter the camera lens. Overall, The project successfully demonstrated that compact, low-cost Internet of Things (IoT) hardware can function independently as an edge-computing node. It proved capable of hosting a local web server, streaming live video data, and calculating complex real-time vision algorithms without needing an expensive external mainframe or processing hub.

Recommendation

As this experiment has few sources of error, we propose some recommendations that can improve reliability of this experiment. First, the ESP32-CAM board and the servo motor should be isolated using dedicated power supplies where the servo must draw power directly from an external 5V source to ensure system stability during rapid tracking movements. Next, connect the ESP32-CAM to a dedicated, offline local Wi-Fi router or personal hotspot instead of a crowded public university network to provide a clean and stable bandwidth channel. Lastly, replace the basic step commands that move the motor at a single fixed speed to a Proportional-Integral-Derivative (PID) algorithm so that the servo's velocity will smoothly decrease as it nears the target face. This advanced mathematical approach completely eliminates robotic, jerky over-corrections, allowing the camera to glide seamlessly into a precise lock.

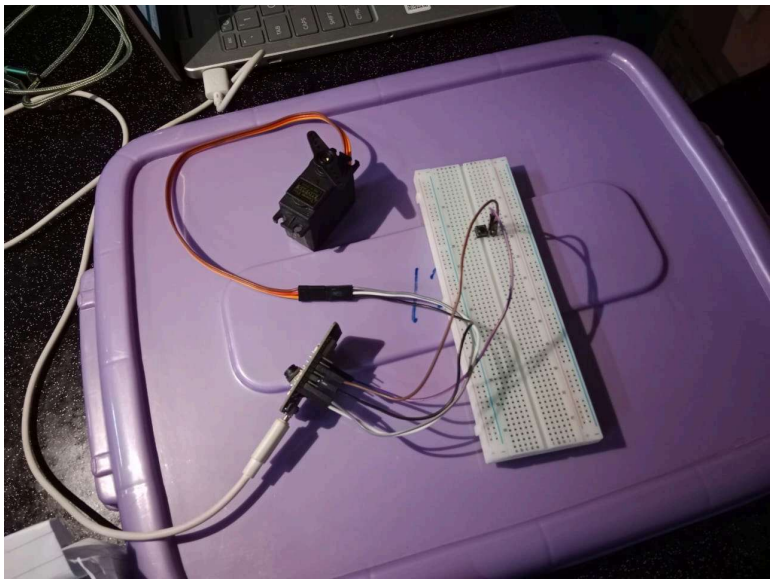
References

Espressif Systems. (2019). ESP32-CAM development board official datasheet and pinout configuration (Version 1.1). Espressif Documentation. https://www.espressif.com/sites/default/files/documentation/esp32-cam_datasheet_en.pdf

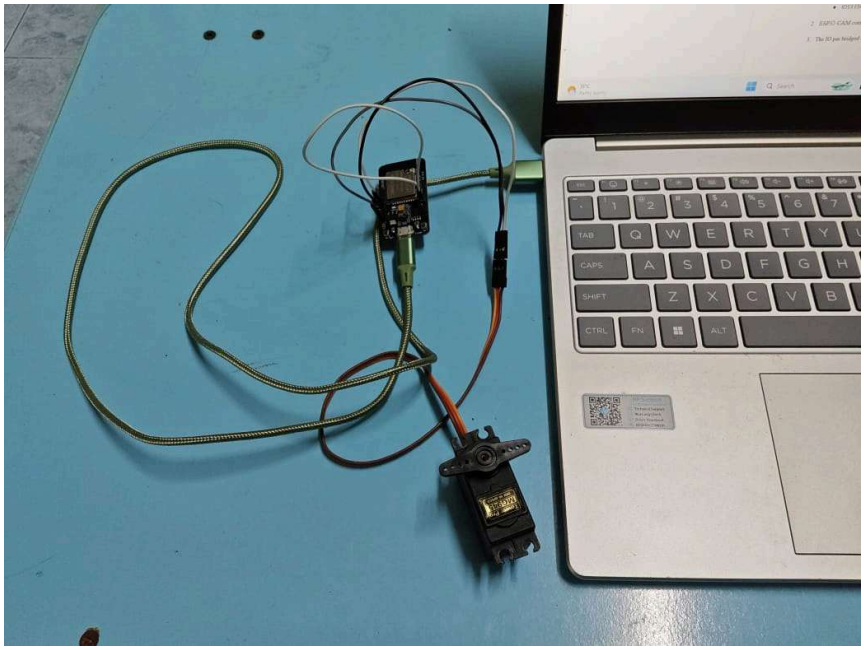
Aithal, S., & Shasthri, S. (2022). Implementation of real-time edge computing surveillance using ESP32-CAM module. *International Journal of Advanced Research in Computer Science*, *13*(2), 45–50. https://www.ijarcs.info/index.php/Ijarcs/article/view/6924

Ramos, J., & Santos, M. (2023). Embedded computer vision: A comparative analysis between standalone AI sensors and edge IoT microcontrollers. *Journal of Mechatronics and Automation Systems*, *7*(3), 112–119. https://doi.org/10.1109/JMAS.2023.102345

Appendices



Appendix 1 : Circuit Setup for Integration of Pushbutton for Manual Control of Servo Panning



Appendix 2 : Circuit Setup for ESP 32 CAM Face detection and Servo Tracking

Code snippet:

```
#include <ESP32Servo.h>

Servo myServo;

int buttonPin = 13;
int servoPin = 14;

int pos = 0;
bool direction = true;

void setup() {
  pinMode(buttonPin, INPUT_PULLUP);
  myServo.attach(servoPin);
}

void loop() {

  int buttonState = digitalRead(buttonPin);


  if (buttonState == LOW) {

    // Servo panning
    if (direction) {
      pos++;
      if (pos >= 180) direction = false;
    } else {
      pos--;
      if (pos <= 0) direction = true;
    }

    myServo.write(pos);
    delay(10);
  }

}
```

Acknowledgement

First and foremost, we would like to express our utmost gratitude to our course instructor, Dr. Zulkifli bin Zainal Abidin, for his invaluable guidance, constant supervision, and constructive feedback throughout the execution of this ESP32-CAM Smart Surveillance System laboratory experiment. His academic insights and technical expertise have deeply enhanced our understanding of embedded edge-computing, wireless networking, and computerized computer vision integration.

We would also like to thank all our group members for their cooperation and teamwork. Everyone contributed their ideas and effort, which made it easier for us to complete all the tasks and overcome the challenges during the lab sessions.

Besides that, we are grateful to our classmates for their help and discussions, which made the learning process more enjoyable and meaningful.

Student's Declaration

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report.** The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us.**

Signature: Read [/]
Name: Nur Irdina Binti Omar Understand [/]
Matric Number: 2312378 Agree [/]
Contribution: Task 1: Materials and equipment, Experimental setup, Methodology, Data collection, Data analysis, and Results

Signature: Read [/]
Name: Muhammad Irsyad Ilham Bin Azizan Understand [/]
Matric Number: 2217555 Agree [/]
Contribution: Abstract, Introduction, Discussion, Conclusion, Recommendation, and Reference

Signature: Read [/]
Name: Nurin Najwa binti Kamaruzaman Understand [/]
Matric Number: 2416554 Agree [/]
Contribution: Task 2: Materials and equipment, Experimental setup, Methodology, Data collection, Data analysis, and Results